Write python OpenCV/Tackpy/DeepLabCut (select the best/easiest library for the case) to analyze video from physics experiment: The experiment: Newton's cradle right ball (there are 5 balls in total) is set off by 45 degreese and set into motion. The background is carton (brown). Balsl are silver. Video is MOV, 4K, 60FPS (700MB), 1:40 min. Ball goes fast at the beginning (little bit blured - but i can change it with filmning slow-mo, if it is a problem) - but i nthe end, all of them are slowly swallowing right and left together I need to detect/measure: 1. Positions of 5 balls 2 Velocities of 5 balls (in m/s) 3 . energies of 5 balls and plot the sum energy over time

2 attachments

Good — I can see the setup clearly. Silver balls against brown cardboard background, very clean contrast. The images show exactly 5 balls (one pulled to the right at 45°). This is perfect for **OpenCV with Hough Circle Transform** — no need for DeepLabCut (overkill for this contrast) or TrackPy (better for microscopy particles). OpenCV's HoughCircles + contour detection on a thresholded frame will work excellently here given the strong silver-vs-brown contrast.

newtons_cradle_tracker.pyPython

Now let me verify the script is syntactically correct and write the requirements file:

requirements_tracker.txtGenerated File

newtons_cradle_tracker.pyPython

requirements_tracker.txtGenerated File

Library choice: OpenCV (plain)

DeepLabCut needs labeled training data (overkill). TrackPy is designed for microscopy particles. Your setup — silver balls on flat brown cardboard with strong contrast — is exactly what **OpenCV's HoughCircles + contour detection** was made for. No training, no GPU, runs on your HP ZBook.

What the script does

Detection pipeline

1. Converts each frame to HSV
2. Masks pixels that are **bright (V > 160) and low-saturation (S < 60)** → isolates silver balls against brown cardboard
3. Runs HoughCircles on the masked greyscale frame
4. Falls back to **contour circularity filter** when Hough struggles (motion blur at high speed)
5. Assigns detections to 5 persistent tracks (nearest-neighbour, sorted left→right)

Physics pipeline

- **Positions** (px → m): auto-calibrated using median detected ball radius vs. --ball_diameter_m

- **Velocities**: Savitzky-Golay derivative (window 21, order 3) of x(t) and h(t) — smooth even at 60 fps
- **Energies**: KE = ½mv², PE = mgh (h measured above equilibrium rest position), E = KE + PE
- **Total energy plot**: stacked KE/PE fill + total E line + exponential decay fit $E(t) = E_0 e^{(-\gamma t)}$ → gives you the damping coefficient γ and time constant τ directly

Outputs (in results/ folder)

File	Content
ball_tracking.csv	frame, time, ball id, x_m, y_m, h_m, vx, vy, speed, KE, PE, E
positions.png	x(t) and h(t) for all 5 balls
velocities.png	speed(t) for all 5 balls
energies.png	per-ball E + **total KE/PE/E** (key physics plot)
energy_decay.png	exponential fit to energy envelope
summary.txt	v_max, E_max per ball + energy retention %

Installation & usage

bash

```
pip install -r requirements_tracker.txt
```

Basic run (4K 60fps, ~66 g balls, 18 cm strings):

bash

```
python newtons_cradle_tracker.py --video experiment.mov
```

If your balls are a different size (measure with a ruler first):

bash

```
python newtons_cradle_tracker.py \
  --video experiment.mov \
  --ball_diameter_m 0.028 \   # measure your specific balls
  --string_length 0.20 \      # pivot to ball center
```

```
  --ball_mass 0.075 \        # weigh one ball
  --skip_frames 2            # process every 2nd frame for speed
```

If slow-mo is not a problem (blur is fine) — use --skip_frames 1 to keep all 60 fps.

Tips for your specific setup

- **Calibration**: measure ball_diameter_m with calipers before filming — this is the only physical calibration needed. A 1 px error at 4K propagates to ~0.3% velocity error.
- **Slow-mo**: if using iPhone/Android slow-mo at 240 fps, pass the actual fps — OpenCV reads it from the file automatically.
- **h_m (height)**: the script uses the first frame's y-position of each ball as the equilibrium reference. So start recording with the cradle at rest for 1–2 seconds before releasing the ball.
- **Frame crop**: if the chrome frame confuses detection (it's also bright + metallic), add --calib_diameter_px 120 (roughly measured in any image viewer) to constrain the Hough radius search.

newtons_cradle_tracker.pyPython

requirements_tracker.txtGenerated File

qustions for setup: I filmed it with iPhone XS Max. Does distances (paralax) make problem? (that ball moving left right is because of projection not as good on the video) - or is it not as impacting?

Great question. Let me think through the physics and geometry carefully.

Parallax — how bad is it?

Short answer: it's minor, not a dealbreaker, but worth understanding.

What's actually happening

The balls swing in a **vertical plane** (like a pendulum). Your iPhone is filming from the front, so the motion you care about (left-right swing) is **in the image plane** — that's the best case for a camera. You don't have depth-direction motion.

The small parallax issue comes from **ball diameter**: the center of a ball that's closer to the camera (swinging toward you slightly, because strings aren't perfectly planar) appears shifted by maybe 1–2 mm in the real world. At 4K from ~0.5–1 m distance that's sub-pixel to a few pixels — negligible for energy plots.

The real distortion you DO have

More impactful than parallax is **perspective foreshortening of the arc**. The balls swing on a circular arc (radius = string length L ≈ 18 cm). What the camera sees is the **projection of that arc onto the image plane**:

$$x_{\text{measured}} = L\sin\ \theta, h_{\text{measured}} = L(1 - \cos\ \theta)$$

This is actually **already correct** — the script uses x and h directly from pixel positions and computes energy from them, so the arc geometry is naturally captured. No correction needed.

What WOULD be a problem

Situation	**Problem?**
Camera slightly off-center (not perfectly frontal)	Small constant offset in x — cancels out in velocity (derivative)
Camera tilted (not level with the pivot bar)	h reference is skewed — **worth leveling the camera**
Balls swinging slightly out-of-plane (toward camera)	~cos(ϕ) correction where ϕ < 5° → error < 0.4%
Rolling/spinning of balls	Not visible, not tracked — irrelevant

Practical recommendation

Two things actually matter more than parallax:

1. **Level your iPhone** — use the built-in level (Camera → level indicator) so the pivot bar of the cradle is perfectly horizontal in frame. This makes h_m accurate.
2. **Film far enough back** — if the cradle spans ~20 cm and you film from 60+ cm, the angular field is small and perspective error is < 1%.

The slow-mo question matters more for velocity accuracy than parallax does. At 60 fps a fast ball (say 0.8 m/s) moves ~13 mm per frame — that's detectable but the Savitzky-Golay smoothing handles it. At **240 fps slow-mo** (iPhone XS Max supports it) each frame is ~3 mm of motion — much cleaner velocity signal. I'd recommend filming in slow-mo if you want accurate velocity/energy curves in the first half of the experiment.

cradle movement spans around 36cm, and the distance from camera to cradle is 35cm

That's a problem.

The geometry is bad

At **35 cm distance** with a **36 cm wide subject**, your half-angle is:

$$\theta = \arctan\left(\frac{18}{35}\right) \approx 27°$$

That's a **very wide viewing angle**. The perspective distortion at the edges is:

$$\cos(27°) \approx 0.89 \Rightarrow \sim 11\% \text{ position error at the edges}$$

For energy that's squared in velocity → **~22% energy error** for the outer balls. That's not negligible at all.

What this means practically

The **center ball (ball 3)** is fine — it's on the optical axis. But **ball 1 and ball 5** at the edges are seen at ~27° off-axis, so their apparent x-positions are compressed relative to true positions. Their velocities and energies will be systematically underestimated.

The fix — two options

Option A: Move the camera back (strongly recommended)

Film from **~120–150 cm** distance. Then:

$$\theta = \arctan\left(\frac{18}{130}\right) \approx 7.9° \Rightarrow \cos(7.9°) \approx 0.99 \Rightarrow < 1\% \text{ error}$$

Use your iPhone's **2× optical zoom** (XS Max has it) to compensate for the larger distance — you get the same framing with proper telephoto geometry. This is the standard technique in sports biomechanics.

Option B: Correct it in software

If you can't re-film, I can add a **perspective undistortion** step to the script. You'd need to provide:

- The known physical width of the cradle frame (measure it in cm)
- 4 corner points of the cradle frame clicked manually in one frame

The script would then apply a homography transform to map pixel coordinates back to true metric coordinates before computing velocities. This corrects the foreshortening exactly.

Recommendation

Re-film with 2× zoom from ~130 cm. It takes 5 minutes and eliminates the systematic error entirely. The script as written will then give you reliable energy curves. Option B (software correction) is doable but adds complexity and requires a careful calibration click in the first frame.

How to install from requiremetns in q comand in bash?

bash

```
pip install -r requirements_tracker.txt
```

If you're on a system where python3/pip3 is explicit:

bash

pip3 install -r requirements_tracker.txt

Or if you want to be safe and install into the correct Python environment:

bash

python -m pip install -r requirements_tracker.txt

i got such an response

image.jpg

That's fine — not an error. The two [notice] lines are just informational:

- **"Defaulting to user installation"** — pip installed the packages into your user folder instead of system-wide. Works perfectly.
- **"A new release of pip is available"** — just a reminder to update pip itself, not related to your packages.

The packages installed successfully. You can now run the script:

bash

python newtons_cradle_tracker.py --video your_video.mov

but i want it to get installed globally - not only in the current folder

You're on Windows (based on the terminal). Run as administrator:

bash

pip install -r requirements_tracker.txt --break-system-packages

Or the cleaner way — open a new terminal **as Administrator** (right-click → "Run as administrator") and then:

bash

pip install -r requirements_tracker.txt

That removes the "site-packages not writeable" restriction and installs globally.

I think, on traiing scale (for developing instruments) (consult ideas) 1. Do HD 30FPS video (+ cropped ad cut) 2. Make black/white background (not just raw carton)?

Good thinking. Let me go through both ideas honestly.

---

1. HD 30fps vs 4K 60fps

Go the opposite direction — keep high fps, reduce resolution.

Setting	Verdict
4K 60fps	Best for physics — high spatial + temporal resolution
HD 30fps	Worse on both axes simultaneously
4K 30fps	Saves file size, acceptable
HD 240fps slow-mo	**Best for fast-moving phase** (first 20s)

The velocity accuracy depends much more on **fps** than resolution. At 30fps a ball moving 0.8 m/s travels ~27mm per frame — that's a large jump and Savitzky-Golay has little data to differentiate cleanly. At 60fps it's ~13mm, at 240fps it's ~3mm.

Practical suggestion: film two clips:

- First 30s at **240fps slow-mo** (captures the fast collision phase)
- Full experiment at **4K 60fps** (captures the slow dying-out phase)

Then process them separately or stitch.

---

2. Black/white background

White is better than cardboard, but not necessary given your contrast.

Looking at your images — the brown cardboard already gives excellent silver-vs-brown contrast. The real enemies are:

Problem	Fix
Chrome frame reflects the background	**Matte black paper** behind the balls only (not whole background) — kills reflections on the frame
Ball surface reflects the frame itself	Can't avoid fully, but matte backing helps
Uneven lighting creates shadows	Diffuse light source (cloudy day / white wall bounce)

Best practical background for OpenCV detection: a sheet of **matte black paper/cardboard** placed directly behind the ball row. Black + silver = maximum contrast, and HoughCircles will lock on

immediately with almost no parameter tuning. White also works but silver balls on white can wash out specular highlights.

Summary recommendation

Change	Worth it?
HD → keep 4K	✅ keep 4K
30fps → 60fps minimum, 240fps slow-mo ideally	✅ strongly yes
Cardboard → matte black behind balls	✅ easy win, 10 min setup
Full white background	⬜ marginal over black

so now how to, in bash move the folder "temp" which is 1 step back in ierarchy of directories into the current irectory (and the same with "results")

bash

```
mv ../temp ./temp
mv ../results ./results
```

.. = one step up. ./ = current folder.

in basz do i always have to type "./" as my current folder? Or onl ywhen i do with somethyng up/down the ierarchy? I just wonder why canot i just type "temp" - not "./temp"?

You can just type temp — ./temp and temp are identical for mv, cp, ls, etc.

bash

```
mv ../temp temp
mv ../results results
```

Works exactly the same.

The only exception where ./ actually matters is when **running an executable**:

bash

```
./main.py    # runs the file
main.py      # bash looks for it in PATH and fails
```

But for file operations (mv, cp, cd, rm) — ./ is optional, just a habit some people have.

so i moved some files - and deleted some - and deleted some folders - how now to "deploy" all the shit to github repository (just liek "erase anything that was there and upload this")

image.jpg

Outlining steps to clean up and deploy files to GitHub